

Goals of CV

1. Measure properties of 2D world
2. Recognise obj, people, emotions
3. Generate: enhance, manipulate existing imgs & videos (computer vision)
4. Organize: mine, search, interact

- ## Difficulties of CV
1. viewpoint variation
 2. illumination
 3. scale
 4. motion
 5. background clutter
 6. intra-class variation

Digital img acquisition

1. Illum-source energy → scene → img system → image plane (discrete sensors) measures amt of energy
2. $P(x,y) = I(x,y) \cdot R(x,y)$
 - $I(x,y)$: illumination incident on the scene
 - $R(x,y)$: reflectance (color) of object in the scene
 - energy proportionate to sensor ADC
3. Exposure Time → amt of light time that incident light can reaching sensor
4. Over-exposure: time too long (white)
5. Under-ex: time too short (dark)
6. Motion blur: time too long obj has moved

Resolution (determine file size)

1. Spatial resolution - or quantization, how many pixels → e.g. 16x16, lower is blurry
2. Intensity resolution - bit depth & gray level (proportionate)
 - 3 distinct gray levels
 - $S = 2^b$, b = bit depth (# bits to represent each pixel)
3. File size (higher resolution → larger file size) + trans time
 - img CMXN
 - $b = k$
 - $M = M \cdot N \cdot k$ → bitmap
 - 8 bits to store 1 comp
4. R, G, B = white, 255, 255, 255
5. S = gray img of each channel: high intensity in that channel → appears white

Colour capture - Bayer Filter

filter by colour, result in patterns = Demosaicing Bayer Pattern:

1. $\begin{bmatrix} R & G & B \\ G & R & G \\ B & G & R \end{bmatrix}$ → borrow from nearest neighbour

2. $\begin{bmatrix} R & G & B \\ G & R & G \\ B & G & R \end{bmatrix}$ → $f(x,y) = f(x,y)$

3. $\begin{bmatrix} R & G & B \\ G & R & G \\ B & G & R \end{bmatrix}$ → $f(x,y) = f(x,y)$

4. $\begin{bmatrix} R & G & B \\ G & R & G \\ B & G & R \end{bmatrix}$ → $f(x,y) = f(x,y)$

5. $\begin{bmatrix} R & G & B \\ G & R & G \\ B & G & R \end{bmatrix}$ → $f(x,y) = f(x,y)$

6. $\begin{bmatrix} R & G & B \\ G & R & G \\ B & G & R \end{bmatrix}$ → $f(x,y) = f(x,y)$

RGB → additive, trichromatic human perception of colour

Grayscale: $I = W_R \cdot R + W_G \cdot G + W_B \cdot B$

Normalized RGB: C, S, B or r, g, b

$r = \frac{R}{R+G+B}$, $g = \frac{G}{R+G+B}$, $b = \frac{B}{R+G+B}$ (intensity)

Intensity carries more info (texture, shades)

HSV colour space

Hue: pure colour (0 to 360)

Saturation: purity, mix white

Value: mix black to white

0 to 255

H = $\begin{cases} \frac{B-R}{V-\min(R,G,B)} \cdot 60^\circ & \text{if } V=R \text{ and } G \geq B \\ \frac{B-R}{V-\min(R,G,B)} \cdot 60^\circ & \text{if } G=V \\ \frac{R-B}{V-\min(R,G,B)} \cdot 60^\circ & \text{if } B=V \\ \frac{R-B}{V-\min(R,G,B)} \cdot 60^\circ & \text{if } V=R \text{ and } R \geq B \end{cases}$

S = $\frac{V-\min(R,G,B)}{V}$ (if $V=\max(R,G,B)$)

Transformations (pt-processing: $x_{ij} = f(p_{ij})$)

1. Brightness: $x_{ij} = p_{ij} + b$ (clipping)
2. Intensity scaling: $x_{ij} = a \cdot p_{ij}$ (contrast)
3. Inverse gamma: $x_{ij} = \frac{p_{ij}}{M}$
4. Gamma mapping: $x_{ij} = 255 \cdot \left(\frac{p_{ij}}{255}\right)^{\gamma}$, $\gamma > 0$
5. $\gamma < 1$: mid-levels, range of dark colours, look brighter
6. $\gamma > 1$: mid-levels, range of bright colours, look darker

Image histogram (gray scale)

Info: global (pt locations) approximate

→ higher count on left: bright

→ higher spread: high contrast

Point process

histogram stretching: $x_{ij} = \frac{p_{ij}-f_1}{f_2-f_1} \times 255$

shift equalisation (non-linear)

forces CDF to be linear

hist: $h_k = \sum_{j=1}^K [p_j - p_{j-1}]$

CDF: $C_k = \sum_{j=1}^K h_j / I$

Linear: $x_{ij} = K \cdot c_{ij}$

Value count % | CDF | new value

Normalisation & equalisation: eq. contrast more, tho expensive

Stretching: eq. of non linear & can change dist even if full range is in use

Histogram thresholding for foreground segmentation

choose right threshold to separate bins

→ Otsu's method - automated thresholding

$T = \arg \min_i W_i(T) \cdot \sigma_i^2(T) + W_b(T) \cdot \sigma_b^2(T)$

$\sigma_i^2(T)$: var of pixels less than or equal to T respectively

$W_i(T)$, $W_b(T)$: # pixels $\leq T$ & $> T$

Filters - Noise Reduction

- Noise: impulses salt and pepper (RV of 0/255)
- Gaussian (var in intensity)
- $R_i = p_{ij} + n$, $n \sim N(0, \sigma^2)$
- Filters (Linear) [no effect to Δ]
- 1. Weighted moving avg: $x_{ij} = \frac{1}{K} \sum_{u=-K/2}^{K/2} \sum_{v=-K/2}^{K/2} p_{i+u, j+v}$
- 2. Correlation (type): $X = P \otimes F$
- 3. Gaussian: $F_{uv} = \text{scale of gaussian}$, σ → choice of σ (var) determines extent of smoothing
- 4. Sharpening: accentuates diff in local avg - enhance edges & fine details but may enhance noise

Filter size & output image

$O = \frac{N-W+2P}{S} + 1$

O : output size, N : input, W : kernel width, P : padding on one side, S : stride

Template Matching

$x_{ij} = \frac{1}{\sum_{u,v} w_{u,v}} \sum_{u,v} f_{u,v} \cdot p_{u,v}$

normalisation removes the influence of mean & scale (response to brighter area vs template match)

Convolution Filter

$X = F * P$

$x_{ij} = \sum_{u,v} f_{u,v} \cdot p_{i-u, j-v}$ (flipped & shifted)

*: img transform, $\otimes$: correlation (similarity)

*: commutative, associative, distributive + scalar factor out, identity

* & $\otimes$: shift invariant

Non-linear filters

Median: good for impulse salt & pepper

→ find median value (avg - sm)

→ mean (continuous)

Edge detection - Gradients

0: no edges; sharp discontinuities in intensity

→ derivatives

grad: vector that pt in the direction of most rapid change in I

$\nabla f = \left[\frac{\partial f}{\partial x}, \frac{\partial f}{\partial y} \right]$ (left to right, up to down)

$\theta = \tan^{-1} \left(\frac{\partial f / \partial y}{\partial f / \partial x} \right)$

Edge strength: $|\nabla f| = \sqrt{\left(\frac{\partial f}{\partial x}\right)^2 + \left(\frac{\partial f}{\partial y}\right)^2}$

discrete finite diff: $f(x) = f(x+1) - f(x)$

Horizontal Sobel: $\begin{bmatrix} 1 & 0 & -1 \\ 2 & 0 & -2 \\ 1 & 0 & -1 \end{bmatrix}$ - S_x

Vertical Sobel: $\begin{bmatrix} -1 & 2 & 1 \\ 0 & 2 & 0 \\ 1 & -2 & 1 \end{bmatrix}$ - S_y

Edge strength: $|\nabla f| = \sqrt{S_x^2 + S_y^2}$

Filter Formula

Gaussian: $G(x,y) = \frac{1}{2\pi\sigma^2} \exp\left(-\frac{x^2+y^2}{2\sigma^2}\right)$

DoG: $\frac{\partial^2 G}{\partial x^2}(x,y)$ (Laplacian of Gaussian)

LoG: $\frac{\partial^2 G}{\partial x^2}(x,y)$ (Laplacian of Gaussian)

Gaussian sigma

σ : scale of G. kernel & determines extent of smoothing

→ σ : larger-scale edge, poor localisation but good detection

Laplacian & finite diff

$\nabla^2 f(x,y) = f(x+1,y) + f(x-1,y) + f(x,y+1) + f(x,y-1) - 4f(x,y)$

img - Laplacian (img) → sharpening effect

has zero-crossing (slice thru 0) → localises edges more accurately but less convenient to use

Canny Edge: NMS - find single pixel line via local max

→ interpolation: keep local max iff $p_{ij} > \text{tanh} \cdot \text{diff of neighs}$

Hysteresis thresholding: E_h & E_L tackle discontinuous edges

Lines

Canny edge gives all edges (lines, curves, noises)

Hough transform: voting technique to gives pt membership to a line, find how many lines, record vote for each possible line passing thru that pt

Math of lines: $Oy = mx + b$, $m = \frac{y_2 - y_1}{x_2 - x_1}$, $b = \frac{y_1 - m x_1}{1}$

Normal line extends from (0,0) to param. line

$\rho = 1 / \sqrt{1/\cos^2\theta + 1/\sin^2\theta}$, dist from (0,0) to param. line

θ : angle between normal line & x-axis

Avg squared dist min E

→ influenced heavily by outliers

Hough Transform: Image space ↔ Param space: $y = mx + b \leftrightarrow y - mx = b$

Robust to outliers → by votes (intersection in param space)

Accumulator array - discretisation of Hough param space to acc. votes

Quantize (range): $-\infty < b \leq +\infty$, $-\infty < m \leq +\infty$

motivated normal parameterised

$y = mx + b \leftrightarrow x \cos \theta + y \sin \theta = \rho$

find ρ, θ | $0 \leq \theta \leq 2\pi$, $0 \leq \rho \leq \max$

positive ρ (version): $x \cos \theta + y \sin \theta = \rho$, $\rho = \rho$

$x \cos(\theta + \pi) + y \sin(\theta + \pi) = -\rho$

trade-off: # bins & precise representation / vote dilution

Param circle

$(x-a)^2 + (y-b)^2 = r^2$ (3D (radius unknown))

circles of diff. radius - normalise by radius (weighted vote)

to prevent large radius dominating the vote → received more complete edge info

robust to occlusion

Gradient info / can both line & C

→ optimise 0:

just 2 votes (single pt) in param space

Voting tips: 0 min irrelevant tokens: consent to edge img

choose good grad / discretization

vote for neighbours (smoothly diff in acc. array)

limit vote extent from each token (p & l)

ant of rastered votes / normalise by radius

Hough T. Pros

Pros: pt processed, robust to occlusion, noise

Cons: search time ↑ w/ model param, spurious peaks, quantization: choice of grid

2D Color Filters $f_{mn} = \frac{1}{2\pi\sigma^2} \exp[-\frac{m^2 + n^2}{2\sigma^2}] \sin[\frac{2\pi \cos W_m + \sin W_n}{\lambda}] + \phi$

m, n - 2D coord of the kernel $\rightarrow \gamma \rightarrow$ aspect ratio of envelop coverage ($\gamma = 0$ else elliptical)

γ - rate of decay of exp envelop (small size of kernel window)

W - orientation of sinusoid in 2D λ - wavelength (inv. proportional to freq, reports)

ϕ - phase shift from center of kernel $\rightarrow$ to so const. # cycles visible / controls alternating light & dark bars $\rightarrow \downarrow \lambda =$ finer, $\uparrow$ freq textures & (vs broader, smoother)

Input	Material classification	Image (Chxw)	Img segmentation	Image (Chxw)
Output	single label material type		segmentation img (Chxw)	
Data sample	(Chxw) img represented as single data pt to be classified		each pixel is a single data pt to be clustered	
Feature representation	texture histogram, where histogramming is done over entire img		1. RGB + (x,y) $\rightarrow$ SLIC 2. spatially smoothed fifteen bins 3. texture histogram (over a window)	
Algo	nearest neighbour classification		clustering also (k-means)	

Segmentation - separate img into coherent regions / objects

$\rightarrow$ clustering (unsup): gp similar data pt to represent them w/ a single token

$\rightarrow$ k-means: iteratively re-assign pt to the nearest cluster center

$\rightarrow$ mean-shift: est. modes of a prob density func.

K-means $\rightarrow$ auto converge but might not be right

- simple, fast, converge to local minima (within clustered scattered error)
- (x) set K, sensitive to initial centers & outliers, detect spherical clusters / less robust to shapes, assumed mean can be computed reasonably & meaningfully
- fragmented segmentations, fuzzy clusters
- chicken & egg prob

1. Random pick K pts from data pts, P_i

2. Iterate until max iter / $GID <$ threshold

$\rightarrow$ assign pts to nearest center: $y_i = \arg \min_j \|P_i - C_j\|$

re-est center: $C_j = \sum_{i \in S_j} P_i / |S_j|$

[mean of pts assigned to S_j]

$\rightarrow$ spatial discontinuities if only color, features intensity

$\rightarrow$ choice of feature space

$\rightarrow$ SLIC built upon k-means

Superspixels $\rightarrow$ use as inputs to various CV algo

$\rightarrow$ gp of pixels sharing common traits

$\rightarrow$ #sp is a hyperparam

$S = \lceil \text{map} / \text{map} \rceil^{1/2}$ initial

Feature $\{r, g, b, x, y\}$

spatial: ds

12 dist measures

composite dist: $D = \lceil \frac{ds_1}{ds_2} \rceil^{1/2}$

References $\leftarrow$ k-means:

- initialise centers on a grid & largest grad per. in 3x3 neighborhood - value common to 5 min pixels $\rightarrow$ cluster convolve
- compute dist (P) only to closest cluster centers (25x25 neighborhood)
- post-processing: replace all sp in each center by avg value $\rightarrow$ to create started class / seg

Filter banks $\rightarrow$ texture representation

- filter & compute statistics e.g. avg, std, 12 dist measures similarity
- interpolate to 2D dimensional feature vector $\rightarrow$ d-filters (multiple)
- 12 dist can simply measure similarity
- but need normalise to prevent one feature dominating dist calc
- norm method: min-max scale or z-score norm

what filters in bank?

- capture diverse physical structures so diff. enough to be in scale (fine details vs broad patterns) horizontal
- collectability to ensure each filter extracts a non-redundant info
- how many $\rightarrow$ trade off between expressive power & computation efficiency
- over cluster to generate combi of filters
- combining sinusoids w/ Gaussian envelop shaped sinusoids
- even ID: $\exp(-\frac{x^2}{2\sigma^2}) \cos(2\pi f x)$
- odd ID: $\exp(-\frac{x^2}{2\sigma^2}) \sin(2\pi f x)$
- odd look like: RGB, Brown $\rightarrow$ Log
- if exp envelop is small compared to λ (wavelength) $\rightarrow$ one oscillation
- high $\rightarrow$ physical dist between 2 consecutive peaks of sine waves is # complete wave cycles per pixel

Textures $\rightarrow$ a physical object textures texture type

- compute
- 1. apply filter bank (training img)
- cluster in feature space & store cluster centers $\rightarrow$ texture dict
- 2. test img: filter img to same bank & get feature rep for each pixel
- assign $\rightarrow$ to nearest cluster (texture) based on feature vector
- Texture = spatial arrangement of each texture ID
- learned texture dict $\rightarrow$ ID to P
- hist of textures: for a region, compute hist of $\rightarrow$ as feat rep.
- vector storing accuracies of each texture
- classify: location (nearest neighbour) for each hist compute a Gaussian to local
- if multiple in smallest dist
- segmentation: get texture map via cluster vectors of filter bank output $\rightarrow$ texture hist in a window
- texture as a feature for segmt.

perceived boundaries (Textures)

- Texture boundary 'line' separates regions of dist texture
- identification of TB:
- texture grad $T(x, y, \theta)$
- split dist into 2 halves by a diameter of orientation
- measure diff in texture between 2 halves by comparing texture hist $\rightarrow$ high dist = E boundaries by all possible orientations

Material classification TextRep

Prerepresent training set $|T| = m$

- filter bank: $m \times h \times w \times d$
- cluster pixel-wise via k-means (each texture cluster: $N \times h \times w \times d$ each label: $E \in \{1, \dots, N\}$)
- texture of hist: M histograms $N \times N$, $(N \times N)$ feature (normalise hist $\rightarrow$ sums of diff sizes $\rightarrow$ total sum = 1)

test img

- Apply filter bank: $h \times w \times d$
- assign textures: $(h \times w \times d)$ bank
- nearest neighbour to assign
- hist of textures: 1 histogram with N bins $\rightarrow 1 \times N$
- classifying: $y_i = \{j, j = \arg \min_k (P_{ij}, f_j)\}$
- $j = 1 \dots N$

Img segment / Text rep

- Input: $h \times w$ image
- Filter bank responses:
- filter bank $\rightarrow$ banked
- (optional) smooth filter output
- cluster: each feature ISD-dimmed
- test on hist (windows):
- filter bank $\rightarrow h \times w \times d$
- cluster pixel-wise filter into N clusters via k-means (cluster texture) $\rightarrow h \times w \times d$ labels
- hist of textures within local window $\rightarrow h \times w \times N$
- each pixel a hist with N bins (w/ local window)
- cluster pixels hist as a feature each $\rightarrow$ is N-dimensional
- but textures hist $\rightarrow$ for segment
- hist not very locally precise
- class. from some window
- better to use specially smoothed texture response

Generalised Hough Transform

1. **Offline modeling**

- model: its boundary pt p & ref pt q.
- tri, compute displacement vector to a & store in table, index
- by $\rightarrow$ info

2. **Detection phase**

- edge pt: use its ID to index stored table $\rightarrow$ retrieve r vectors to vote for ref pt.

How vote for detection:

- index by matched local patterns instead of θ
- off model via voting $\rightarrow$ let features
- clusters / vectors vot for all competitive models $\rightarrow$ output model points that receive a lot of votes

Diagram: $\sum_{i=1}^N \frac{1}{N} \sum_{j=1}^N \frac{(x_j - x_i)^2 + (y_j - y_i)^2}{N} + N |g|$

cross comp $\rightarrow$ test set

flatten into $h \times w$ vector

hg: # pixels in filter g

X: $N \times hg$ data matrix ($N \times hg$)

Y: $N \times hg$ output (res), vectorised

Gaussian

kernel g results in sharp Gaussian over mult edges

x_i - frame 1, frame i - x_i

$S = (X^T X + \lambda I)^{-1} X^T Y$

$\rightarrow$ use $\hat{g} = F(S) \rightarrow$ momental update

Scanned with CamScanner

